The assignment set consists of 5 pages

KRISTIANIA
PG3302 Software Design
Exam/semester assignment 2026

Implementation

- The assignment is carried out individually or in groups of two participants.
- If you submit as a group, every participant is responsible for the complete submission.
- An individual oral examination is a mandatory part of the assessment. The written submission and the oral examination are assessed together.
- The solution must be written in C# and use .NET 10.
- All written course material is permitted, including slides, the textbook, your own work, and exercise solutions.
- You may ask the course teacher for clarification, but you may not receive solution-specific help from people outside your group.
- Expected scope: approximately 37,5 hours (1 week) of work per participant. The work must be distributed across the assignment period and documented through the repository history.

Use of artificial intelligence and external material

The purpose of the examination is to assess your own knowledge and skills. Generative AI must not be used to produce substantial parts of the software, tests, design, report, or reflections on your behalf. Small IDE-integrated completion suggestions are permitted, but you remain fully responsible for every submitted line and must be able to explain and modify it during the oral examination.

- Disclose any use of generative AI beyond ordinary autocomplete in the source and assistance log described below.
- Identify all externally sourced or adapted code, diagrams, text, tutorials, and example solutions.
- Undisclosed external assistance or ready-made material may be handled as an examination irregularity under the applicable regulations.

Important: The assessors will not attempt to determine authorship from writing style or code style alone. Authorship and competence are established through development evidence and the individual oral examination.

Required delivery

The hand-in must contain all three delivery components below. The oral examination is an additional mandatory assessment component.

1. **A PDF report** documenting the solution, development process, and evidence of learning.
2. **A Git repository** containing the complete project and its genuine development history.
3. **A ZIP archive** of the source code and all files needed to build, test, and run the submitted version.

1. PDF report

The report is an evidence guide to the submitted solution, not a traditional theory report. It must be no more than 6 pages, including diagrams and screenshots. A source and AI-assistance list may use 1 additional page. (Separate cover page or table of contents not required.) Do not include any appendices. Code screenshots are normally unnecessary because the assessor has access to the repository. Refer instead to file paths, class names, tests, and selected Git commit IDs.

Include the following: (length per part is meant as guidance, to stay below the total max length of 6 + 1 pages)

- **System overview** - maximum 1/2 page: Describe the chosen domain, intended users, and core functionality.
- **UML and architecture** - maximum 1 1/2 pages: Include an early class diagram and a final class diagram. Briefly explain the main architectural change between them and how the final diagram relates to the layers and implementation.
- **Design and implementation** - maximum 1 1/2 pages: Identify the most important object-oriented design choices, layering, persistence approach, relevant SOLID or GRASP principles, implemented design patterns, and one meaningful refactoring. Refer to the relevant files/classes and before/after commits.
- **Testing** - maximum 1 page: Describe the unit and integration tests, what they cover, what they do not cover, and why those areas were selected.
- **Development process, Git, and contribution** - maximum 1 page: Explain how the work developed over time. Highlight no more than five significant commits. For groups of two, state each participant's contributions and describe at least one concrete collaboration practice, such as pair programming, code review, or a pull request.
- **Known limitations and technical debt** - maximum 1/2 page: List known errors, limitations, shortcuts, and the improvements you would prioritise next.
- **Sources and assistance log** - maximum 1 extra page: List external sources and all non-trivial AI assistance. For each AI use, name the tool, purpose, affected files/sections, and how the output was checked and changed.

Include a compact evidence table with at least these columns:

Syllabus area	What the submission demonstrates	File/class/test	Commit ID(s)
UML evolution and architecture	Brief, solution-specific evidence	Path or symbol	Short hash(es)
Design principles and patterns	Brief, solution-specific evidence	Path or symbol	Short hash(es)
Testing	Brief, solution-specific evidence	Path or symbol	Short hash(es)
Refactoring	Brief, solution-specific evidence	Path or symbol	Short hash(es)

2. Git repository and development history

The repository is part of the assessed submission, not merely a transport mechanism for the final code. It must show how the solution developed over time.

- **Create the repository near the beginning of the assignment period** and give the assessors access no later than the submission deadline.
- Use a separate, identifiable Git account for each participant. Shared accounts do not demonstrate individual contribution.
- Each participant must make at least 10 meaningful commits, distributed across at least 6 separate calendar dates and across the early, middle, and final phases of the project.

- Use concise commit messages that describe the change. A meaningful commit represents a coherent development step, not a mechanically split or empty change.
- Commit design material, tests, and report-related evidence when appropriate, not only the final source files.
- Groups of two must make both participants' contributions visible and must also be explained in the report.
- Do not fabricate, backdate, squash, or rewrite the history for submission. A late bulk import does not satisfy the continuous-development requirement.
- Highlight no more than five significant commits in the report. The assessors may inspect other commits when needed.
- Add a README with prerequisites, the solution-file location, project structure, diagram location, and short instructions for opening, building, running, and testing the solution.
- Tag the submitted commit as exam-submission. The ZIP archive and report must correspond to that tagged version.

3. Source-code archive

Submit a ZIP archive of the repository at the exam-submission tag (excluding generated build folders such as bin and obj, if you like). Include the solution and project files, source code, tests, data files, migrations or schema files, and everything else required to reproduce the submitted result. Do not include machine-specific configuration.

- Include a standard solution file so the complete solution can be opened directly in an IDE.
- The supported course IDEs are Visual Studio on Windows and Rider on Windows or macOS. Visual Studio Code is also permitted.
- The solution must build and run using .NET 10 in the IDE used by the group, following the short instructions in the README.
- The automated tests must be discoverable and runnable using the IDE's normal test interface.
- NuGet packages are permitted and fine. On the other hand, the assessor must not need to manually install tools or services.
- Persistent storage must contain sufficient sample data for the assessor to test the application's main functions.

Individual oral adjustment

Each participant will attend an individual oral adjustment based on the submitted report, repository, and software. The assessors select the areas to discuss; students do not choose the parts they personally prefer.

You may be asked to:

- explain how a selected feature moves through the layers and relate the code to the UML class diagram;
- explain a selected class, interface, design pattern, SOLID (or GRASP) decision, or refactoring;
- trace and justify selected Git commits, including how your own contribution developed;
- run and explain selected unit or integration tests;
- debug a bounded problem or make and explain a small modification to the submitted solution; and
- reflect on alternative designs, technical debt, limitations, collaboration, and lessons learned.

Assessment consequence: If a participant cannot independently explain and work with the submitted solution, the assessors cannot treat that material as evidence of that participant's achievement of the learning outcomes. Individual oral performance affects the individual grade.

Requirements regarding content

Design and implement a coherent software solution in C# using .NET 10. Choose a domain with enough data and behaviour to demonstrate the requirements naturally. A traditional business-to-consumer system is often suitable. Games are not recommended because they commonly make persistence, CRUD, layering, and the full breadth of the syllabus harder to demonstrate.

Architecture and project structure

- Use a clear three-layer architecture: presentation, domain/application logic, and data access/persistence.
- Provide a **console-based** user interface that is logical, usable, and robust. Graphical appearance is not assessed.
- Use object-oriented structure with appropriate classes, properties, interfaces, inheritance and/or composition.
- The hand-in must contain at least a C# application project and an NUnit test project. Additional class-library projects may be used to make architectural boundaries explicit.
- Target net10.0 in every submitted project unless a framework-specific reason is documented and approved in advance.

Functional and persistence requirements

- Implement meaningful create, read, update, and delete operations for the chosen domain.
- Persist data using SQLite with Entity Framework Core or ADO.NET, or use JSON/XML when that choice is justified by the domain and design.
- Separate persistence concerns from domain logic and presentation concerns.
- Include sample data and a reproducible setup so the application can be evaluated immediately.

Software-design requirements

- Create and preserve at least two UML class diagrams from different stages of development. They must accurately represent important classes, relationships, multiplicities, and architectural boundaries at their respective stages.
- Briefly document the main change between the diagrams and relate the final diagram to the submitted code.
- Apply layering and relevant SOLID or GRASP principles. Course teaching emphasises SOLID, but well-justified use of GRASP is equally acceptable. Be able to identify both good decisions and remaining weaknesses.
- Implement at least two recognisable design patterns from the course where they solve genuine design problems. Pattern names without meaningful implementation do not count.
- Carry out at least one meaningful refactoring during development and preserve evidence of the before-and-after structure in Git.
- Use the IDE's build, test, refactoring, and debugging tools as part of the development process.

Testing requirements

- Implement meaningful NUnit unit tests for selected domain/application logic, including normal cases and relevant edge cases.
- Implement at least one integration test that crosses a real boundary between important parts of the solution. Mocks or fakes may be used where appropriate, but must be justified.
- Tests are not expected to cover every line, but the selection must demonstrate breadth, reasoning, and an understanding of test limitations.

Assessment

The complete submission and oral examination are assessed holistically. Particular emphasis is placed on:

- the correctness and usability of the implemented solution;
- C#/.NET proficiency and object-oriented code quality;
- UML, architecture, layering, design principles, design patterns, and justified design choices;
- testing, refactoring, and the ability to evaluate and improve existing code;
- a genuine, traceable development process and effective collaboration;
- the quality and verifiability of the report, repository, sources, and assistance log; and
- each participant's independent understanding demonstrated during the oral examination.

Quality over checklist completion: Features included only to mention a syllabus term, without a coherent purpose or understanding, may weaken rather than strengthen the submission. Explain any deliberately simplified or educational design choice.

- End of task set -